Practical 11: Responsive Web Pages using HTML, CSS and JavaScript

Aim

To design and implement a responsive web page using HTML, CSS, and JavaScript.

Theory

HTML (HyperText Markup Language) provides the semantic structure and content of a web page using nested elements (tags) such as div, h2, p, and button. CSS (Cascading Style Sheets) controls the presentation of that structure, including layout, colors, spacing, fonts, and visual effects like box-shadow and border-radius, and can be applied inline, via an internal <style> block, or via an external stylesheet. JavaScript adds interactivity and dynamic behavior to an otherwise static page by manipulating the Document Object Model (DOM) - the browser's in-memory tree representation of the HTML - at runtime, for example by changing an element's innerText or innerHTML in response to user actions. Responsive Web Design ensures a page's layout automatically adjusts to fit different screen sizes and devices (desktop, tablet, mobile) without requiring separate versions of the site, commonly achieved using relative units (%, em, rem) instead of fixed pixels, flexible containers (Flexbox/Grid), media queries, and constraints like max-width so content neither overflows small screens nor stretches awkwardly on large ones. JavaScript event handlers, such as the onclick attribute or addEventListener(), allow the page to respond dynamically to user actions like clicks, input, or form submission without requiring a full page reload.

Algorithm

1. Create the HTML structure with a container div styled as a 'card'.

2. Use CSS to style the body and card with background color, padding, border-radius, and box-shadow, and set a max-width on the card for responsiveness.

3. Add a button element with an onclick JavaScript handler.

4. On button click, use JavaScript to update the innerText of a paragraph element to show a message.

5. Open the page in a browser and verify the layout adjusts to screen width and the button click updates the message.

Program

```
<!DOCTYPE html>
<html>
<head>
<style>
  body { font-family: Arial; background: #f0f2f5; margin: 0; }
  .card { max-width: 400px; margin: 40px auto; padding: 20px;
          background: white; border-radius: 10px; box-shadow: 0 2px 8px #ccc; text-align: center; }
  button { padding: 8px 16px; background: #4CAF50; color: white; border: none; border-radius: 5px; }
</style>
</head>
<body>
  <div class="card">
    <h2>Responsive Card</h2>
    <p>This page resizes with the screen.</p>
    <button onclick="document.getElementById('msg').innerText='Button Clicked!'">Click Me</button>
    <p id="msg"></p>
  </div>
</body>
</html>
```

Output

```
Browser renders a centered white card on a light grey
```

```
background with heading "Responsive Card", a paragraph,
and a green "Click Me" button.

On clicking the button, the page updates and shows:
  Button Clicked!

The card layout adjusts automatically to fit any
screen width (responsive design).
```

Result / Conclusion

A responsive web page was successfully designed and implemented using HTML, CSS, and JavaScript, with verified interactivity and layout adjustment.

Viva Questions

Q1. What is the difference between inline, internal, and external CSS?

Ans: Inline CSS: style attribute directly on an HTML element (<p style='color:red'>). Highest specificity, hardest to maintain. Internal CSS: <style> block inside the <head> — applies to one page, good for small prototypes. External CSS: a separate .css file linked via <link rel='stylesheet'>. Best practice for real projects — separates concerns, enables caching, shared across multiple pages. Specificity order: inline > ID > class > tag.

Q2. What is the Document Object Model (DOM)?

Ans: The DOM is a tree-structured, in-memory representation of an HTML document created by the browser when a page loads. Each HTML element becomes a node in the tree. JavaScript can access and manipulate this tree via the document object (e.g., document.getElementById(), element.innerText = '...'). Changes to the DOM are immediately reflected in the browser without reloading the page, enabling dynamic interactivity.

Q3. What is responsive web design and what techniques achieve it?

Ans: Responsive web design (RWD) makes a webpage layout adapt to different screen sizes (desktop, tablet, mobile) automatically. Key techniques: (1) fluid grids using % widths instead of fixed px; (2) flexible images (max-width: 100%); (3) CSS media queries (@media (max-width: 768px) {...}) to apply different styles at breakpoints; (4) CSS Flexbox and Grid for adaptive layouts; (5) the viewport meta tag (<meta name='viewport' content='width=device-width, initial-scale=1'>).

Q4. What is the difference between innerText and innerHTML?

Ans: innerText sets or gets the visible text content of an element, treating the value as plain text — any HTML tags in the string are escaped and displayed literally. innerHTML sets or gets the HTML markup inside an element — HTML tags are parsed and rendered by the browser. Use innerText for plain text to avoid XSS vulnerabilities; use innerHTML only when you deliberately want to insert HTML markup and have sanitised the input.

Q5. What is the CSS Box Model?

Ans: The CSS Box Model describes how each HTML element is rendered as a rectangular box consisting of four layers from inside out: Content (the actual text/image), Padding (space between content and border, transparent), Border (visible line around the padding), Margin (transparent space outside the border, separating elements). The default box-sizing is content-box (width/height apply to content only); box-sizing: border-box includes padding and border in the width/height calculation, which is more intuitive for layout.